Лабораторна робота № 2. Оркестрація та хореографія мікросервісів.

Мета роботи: вивчити структуру мікросервісного додатку та принцип комунікації сервісів у ньому. Створити власну мікросервісну систему для демонстрації керованої взаємодії сервісів.

Задача: на основі прикладу, описаного у Лабораторній роботі №1, створити власну мікросервісну систему. Необхідно розгорнути кілька мікросервісів і показати, що вони між собою взаємодіють синхроно (наприклад клієнт зробив запит з якимось рядком, а кожен з сервісів до нього щось допише, головне показати протоколи спілкування) та асинхронно (з допомогою меседж брокеру типу RabbitMQ чи event store aka Kafka). Налаштувати автоматизоване розгортання всіх контейнерів, продемонструвати механізм горизонтального масштабування.

Завдання

1. Для кожного контексту з застосунку, описаного у попередній роботі, власний мікросервіс (щонайменше 3).

2. Реалізувати синхронну взаємодію 2х чи більше сервісів (один викликає інший). Підготувати опис кроків, які було зроблено задля забезпечення діскавері.

3. Підключити інфраструктурні сервіси (edge/api gateway, discovery, and so on)

4. Протестувати роботу власного сервісу та проілюструвати це. Підняти декілька інстансів одного сервісу, показати механізм горизонтального масштабування і пояснити механізми балансування.

5. Обрати message broker (рекомендовано Amazon SQS або RabbitMQ для простоти)

6. Реалізувати асинхронну комунікацію. При створенні сутності чи її модифікації повідомлення повинно падачи в чергу. Створити сервіс-консьюмер, що споживатиме повідомлення з черги.

6. Вивчити основні підходи до мікросервісної архітектури.

Хід роботи:

Рекомендовано ознайомитись з best practices створення докерфайлів за посиланням:

https://docs.docker.com/build/building/best-practices/

Детальна інформація по всіх інструкціях докерфайлу доступна за посиланням https://docs.docker.com/engine/reference/builder/

Розглянемо типову структуру докерфайлу:

```
FROM ubuntu:15.04
COPY . /app
RUN make /app
CMD python /app/app.py
```

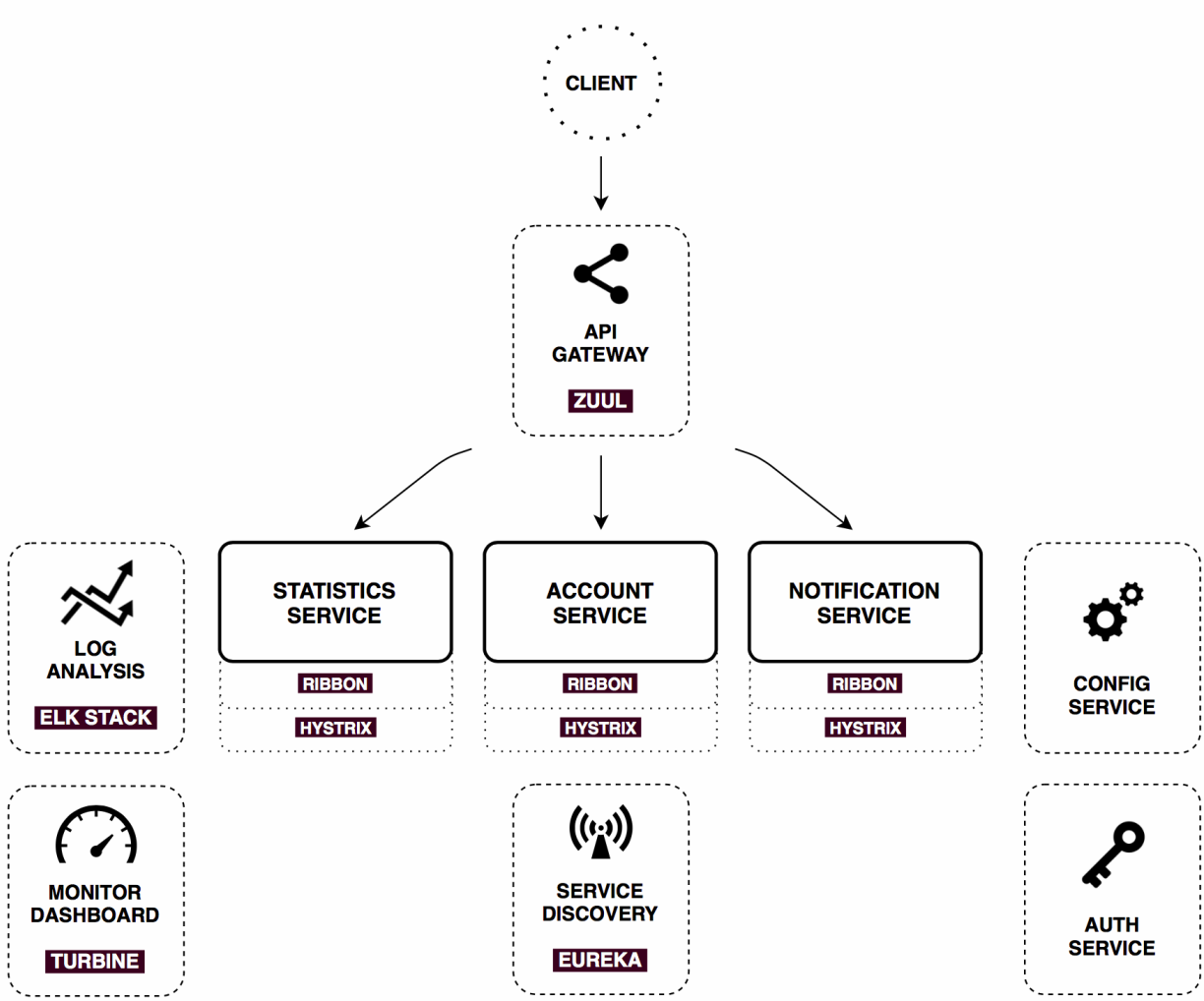
Кожна інструкція створює новий docker-layer (котрий кешується):

- FROM створює layer з базового образу ubuntu:15.04.
- COPY додає файли з поточної лиректорії клієнта (також є команда ADD, котра інкапсулює роботу з архівами і веб-ресурсами).
- RUN виконує наступну команду, в даному випадку білд за допомогою make.
- CMD команда, котру виконує контейнер.

Ознайомитись з реалізацією мікросервісних архітектур. Презентація: https://goo.gl/HFduA6

https://github.com/sqshq/PiggyMetrics – репозиторій з якісно реалізовани прикладом на стекові Spring Cloud Netflix

https://cloud.spring.io/spring-cloud-netflix/



Компонентний приклад реалізації [https://github.com/sqshq/PiggyMetrics]

Основні компоненти мікросервісної архітектури

1. Discovery service – реєстр сервісів в системі
2. Edge service – API шлюз
3. Load balancers – балансувальники навантаження між активними інстансами сервісів, бувають client-side, server side
4. Config service
5. Circuit breaker – призначений для мінімізації негативного ефекту від запитів до “поламаних” сервісів , ізолюючи їх.

Комунікація між сервісами

В дані лабораторній роботі необхідно налаштувати комунікацію між мікросервісами інкапсульованими в докер контейнери, для цього в навчальних цілях можна використовувати docker-compose, на продакшен серидовищах краще користуватись k8s https://kubernetes.io чи docker-swarm

Нижче наведено простий приклад compose-файлу:

```
eddie:
  restart: always
  build: ../eddie/
  links: #задаємо резолв адрес між контейнерами
  - calculator
  - api
  ports: #мапінг портів
  - 8080:8080
  extra_hosts: #додаємо зовнішні сервісі до хостів
  - "extra_host:external.service.com"
  env_file: #вичитуємо змінні серидовища доступні в контейнері
  - local_var.env


calculator:
  restart: always
  build: ../calculator/
  links:
  - minio
  ports:
  - 8083
  extra_hosts:
  - "extra_host:external.service.com"
  env_file:
  - local_var.env


api:
```

Також для створення мережі можна використовувати networks замість links

Детальний опис доступний за посиланням
https://docs.docker.com/compose/compose-file/#compose-file-structure-and-examples

Масштабування можна дослідити за допомогою docker-compose scale:

```
docker-compose scale web=2 worker=3
```

Ознайомтеся з патернами CQRS та Event sourcing

https://martinfowler.com/bliki/CQRS.html

https://martinfowler.com/eaaDev/EventSourcing.html

Контрольні питання:

6. Питання по основним командам докерфайлу.
7. Що таке сервіс-орієнтована архітектура.
8. Що таке мікросервісна архітектура, які її основні ознаки і як відрізнити від звичайної СОА.
9. Domain Driven Design (DDD) і його зв’язок з мікросервісними архітектурами.
10. Розповісти про призначення основних компонентів мікросервісної архітектури.
11. Зобразити архітектуру інтернет магазину (чи іншого сервісу) в термінах мікросервісів, враховуючі інфраструктурні компоненти.
12. Розповісти про основні патерни проектування мікросервісних архітектур.
13. Orcherstration vs Choreography.
14. Описати реалізацію паттерну Circuit breaker.
15. Описати базові команди docker-compose, або іншого пакету, який застосовувався в роботі.
16. Описати механізм знаходження мікросервісів одне одним, як відбувається їх горизонтальне масштабування.
17. Роль Service discovery та принцип роботи балансувальника на обраній технології при горизонтальному масштабуванні.